

Unidad 3: Métodos de Colecciones y Operadores Básicos

Introducción a las Colecciones

En Python, las colecciones son estructuras de datos que permiten almacenar y manipular grupos de elementos. Las tres colecciones más utilizadas son listas, conjuntos y diccionarios. Cada una de estas colecciones tiene sus propios usos y métodos específicos para operar sobre los datos que contienen.



Listas

Las listas son colecciones ordenadas y mutables de elementos. Los elementos dentro de una lista pueden ser de cualquier tipo de dato y las listas permiten la duplicación de elementos.

Métodos básicos para listas:

- `append(x)`: Añade un ítem al final de la lista.
- `extend(iterable)`: Extiende la lista agregando todos los ítems de un iterable.
- `insert(i, x)`: Inserta un ítem en una posición dada.
- `remove(x)`: Elimina el primer ítem de la lista cuyo valor sea x.
- `pop([i])`: Elimina el ítem en la posición dada de la lista y lo devuelve.
- `clear()`: Elimina todos los elementos de la lista.
- `index(x, [start, end])`: Devuelve el índice del primer ítem cuyo valor sea x.
- `count(x)`: Devuelve el número de veces que x aparece en la lista.

- ``sort(key=None, reverse=False)``: Ordena los ítems de la lista (los argumentos pueden ser usados para personalizar el orden).
 - ``reverse()``: Invierte los elementos de la lista.
 - ``copy()``: Devuelve una copia superficial de la lista.
-

Conjuntos

Los conjuntos son colecciones no ordenadas y no indexadas. Cada ítem en un conjunto es único, lo que significa que no puede haber duplicados.

Métodos básicos para conjuntos:

- ``add(x)``: Añade un ítem al conjunto.
 - ``remove(x)``: Elimina un ítem del conjunto. Genera un error si el ítem no existe.
 - ``discard(x)``: Elimina un ítem del conjunto si existe.
 - ``pop()``: Elimina y devuelve un ítem arbitrario del conjunto.
 - ``clear()``: Elimina todos los elementos del conjunto.
 - ``union(*others)``: Devuelve la unión de conjuntos.
 - ``intersection(*others)``: Devuelve la intersección de conjuntos.
 - ``difference(*others)``: Devuelve la diferencia de conjuntos.
 - ``symmetric_difference(other)``: Devuelve la diferencia simétrica de conjuntos.
 - ``issubset(other)``: Verifica si el conjunto es un subconjunto de otro.
 - ``issuperset(other)``: Verifica si el conjunto es un superconjunto de otro.
 - ``copy()``: Devuelve una copia superficial del conjunto.
-

Diccionarios

Los diccionarios son colecciones no ordenadas de pares clave-valor. Cada clave en un diccionario debe ser única y los valores pueden ser de cualquier tipo de dato.

Métodos básicos para diccionarios:

- ``get(key[, default])``: Devuelve el valor de la clave. Si la clave no existe, devuelve el valor por defecto.
- ``keys()``: Devuelve una vista de las claves en el diccionario.
- ``values()``: Devuelve una vista de los valores en el diccionario.
- ``items()``: Devuelve una vista de los pares clave-valor en el diccionario.
- ``pop(key[, default])``: Elimina la clave y devuelve su valor. Si la clave no existe, devuelve el valor por defecto.
- ``popitem()``: Elimina y devuelve un par clave-valor aleatorio del diccionario.
- ``clear()``: Elimina todos los ítems del diccionario.

- `update([other])`: Actualiza el diccionario con los pares clave-valor de other, sobrescribiendo las claves existentes.
 - `setdefault(key[, default])`: Si la clave está en el diccionario, devuelve su valor. Si no, inserta la clave con el valor por defecto y devuelve el valor por defecto.
-

Resumen

Python proporciona diversas colecciones como listas, conjuntos y diccionarios, cada una con sus propios métodos específicos. Las listas son ordenadas y permiten duplicados, los conjuntos no están ordenados y no permiten duplicados, y los diccionarios almacenan pares clave-valor con claves únicas. Los métodos asociados a cada tipo de colección permiten la manipulación eficiente de los datos según las necesidades del programa.

Métodos de Cadenas en Python

Descripción detallada de los métodos de cadenas

Python proporciona una variedad de métodos para manipular y trabajar con cadenas de caracteres. A continuación se describen los métodos más comunes con ejemplos prácticos de uso en código.

`upper()`

Convierte todos los caracteres de la cadena a mayúsculas.

Ejemplo:

```
python
cadena = "Hola Mundo"
print(cadena.upper())

'''
```

Salida:

```
'''
HOLA MUNDO

'''
```

`lower()`

Convierte todos los caracteres de la cadena a minúsculas.

Ejemplo:

```
```python
cadena = "Hola Mundo"
print(cadena.lower())
```

```
```
```

Salida:

```
```
hola mundo
```

```
```
```

`capitalize()`

Convierte el primer carácter de la cadena a mayúscula y el resto a minúsculas.

Ejemplo:

```
```python
cadena = "hola mundo"
print(cadena.capitalize())
```

```
```
```

Salida:

```
```
Hola mundo
```

```
```
```

`title()`

Convierte el primer carácter de cada palabra a mayúscula y el resto a minúsculas.

Ejemplo:

```
```python
cadena = "hola mundo"
print(cadena.title())
```

```
'''
```

**Salida:**

```
'''
```

Hola Mundo

```
'''
```

---

## **count()**

Cuenta cuántas veces aparece una subcadena en la cadena.

**Ejemplo:**

```
'''python
cadena = "Hola mundo, hola universo"
print(cadena.count("hola"))
```

```
'''
```

**Salida:**

```
'''
```

1

```
'''
```

---

## **find()**

Devuelve el índice de la primera aparición de una subcadena en la cadena. Si no se encuentra, devuelve -1.

**Ejemplo:**

```
'''python
cadena = "Hola mundo"
print(cadena.find("mundo"))
```

```
'''
```

**Salida:**

```
'''
```

5

```
'''
```

---

## ``rfind()``

Devuelve el índice de la última aparición de una subcadena en la cadena. Si no se encuentra, devuelve -1.

### **Ejemplo:**

```
'''python
cadena = "Hola mundo, hola universo"
print(cadena.rfind("hola"))
```

```
'''
```

### **Salida:**

```
'''
```

```
12
```

```
'''
```

---

## ``split()``

Divide la cadena en una lista utilizando el separador especificado.

### **Ejemplo:**

```
'''python
cadena = "Hola mundo"
print(cadena.split())
```

```
'''
```

### **Salida:**

```
'''
```

```
['Hola', 'mundo']
```

```
'''
```

---

## `join()`

Une una lista de cadenas utilizando la cadena como separador.

### Ejemplo:

```
```python
lista = ["Hola", "mundo"]
print(" ".join(lista))
```

```
```
```

### Salida:

```
```
Hola mundo
```

```
```
```

---

## `strip()`

Elimina los caracteres especificados (por defecto, espacios) desde el principio y el final de la cadena.

### Ejemplo:

```
```python
cadena = "  Hola mundo  "
print(cadena.strip())
```

```
```
```

### Salida:

```
```
Hola mundo
```

```
```
```

---

## `replace()`

Reemplaza todas las apariciones de una subcadena por otra subcadena.

### Ejemplo:

```
```python
```

```
cadena = "Hola mundo"
print(cadena.replace("mundo", "universo"))

...
```

Salida:

```
...
Hola universo
...
```

Métodos de Listas

Métodos de Listas en Python

Python proporciona varios métodos para manipular y modificar listas de manera eficiente. A continuación se explican algunos de los métodos más comunes junto con ejemplos prácticos de uso en código.

``clear()``

El método ``clear()`` elimina todos los elementos de una lista, dejándola vacía.

Ejemplo:

```
```python
letras = ['a', 'b', 'c', 'd', 'e']
letras.clear()
print(letras)

...
```

**Salida:**

```
...
[]
...
```

---



## **`extend()`**

El método ``extend()`` añade todos los elementos de un iterable (lista, tupla, etc.) al final de la lista actual.

### **Ejemplo:**

```
```python
numeros = [1, 2, 3]
numeros.extend([4, 5, 6])
print(numeros)

```
```

### **Salida:**

```
```
[1, 2, 3, 4, 5, 6]
```
```

---

## **`insert()`**

El método ``insert()`` inserta un elemento en una posición específica de la lista. El primer argumento es el índice en el que se insertará el elemento y el segundo argumento es el elemento a insertar.

### **Ejemplo:**

```
```python
numeros = [1, 2, 3, 4]
numeros.insert(2, 'a')
print(numeros)

```
```

### **Salida:**

```
```
[1, 2, 'a', 3, 4]
```
```

---

## **`reverse()`**

El método ``reverse()`` invierte el orden de los elementos en la lista.

### Ejemplo:

```
```python
numeros = [1, 2, 3, 4, 5]
numeros.reverse()
print(numeros)
```
```

### Salida:

```
```
[5, 4, 3, 2, 1]
```
```

---

## `sort()`

El método `sort()` ordena los elementos de la lista en orden ascendente por defecto. Se puede usar el argumento `reverse=True` para ordenar en orden descendente.

### Ejemplo:

```
```python
numeros = [5, 2, 9, 1, 5, 6]
numeros.sort()
print(numeros)
```
```

### Salida:

```
```
[1, 2, 5, 5, 6, 9]
```
```

## Ejemplo con `reverse=True`:

```
```python
numeros = [5, 2, 9, 1, 5, 6]
numeros.sort(reverse=True)
print(numeros)
```
```

**Salida:**

```
...
[9, 6, 5, 5, 2, 1]
...
```

---

## Métodos de Conjuntos en Python

### Descripción de los métodos de conjuntos

Los conjuntos en Python son colecciones desordenadas de elementos únicos. Python proporciona varios métodos para manipular y trabajar con conjuntos. A continuación se describen algunos de los métodos más comunes junto con ejemplos prácticos de uso.

#### ``copy()``

El método ``copy()`` devuelve una copia superficial del conjunto.

**Ejemplo:**

```
```python  
set1 = {1, 2, 3}  
set2 = set1.copy()  
print(set2)  
...
```

Salida:

```
...  
{1, 2, 3}  
...
```

``isdisjoint()``

El método ``isdisjoint()`` devuelve ``True`` si dos conjuntos no tienen elementos en común.

Ejemplo:

```
```python  
set1 = {1, 2, 3}
set2 = {4, 5, 6}
print(set1.isdisjoint(set2))
...
```

```
'''
```

**Salida:**

```
'''
```

True

```
'''
```

---

## **`issubset()`**

El método ``issubset()`` devuelve ``True`` si todos los elementos del conjunto están en otro conjunto.

**Ejemplo:**

```
'''python
set1 = {1, 2}
set2 = {1, 2, 3, 4}
print(set1.issubset(set2))
```

```
'''
```

**Salida:**

```
'''
```

True

```
'''
```

---

## **`issuperset()`**

El método ``issuperset()`` devuelve ``True`` si el conjunto contiene todos los elementos de otro conjunto.

**Ejemplo:**

```
'''python
set1 = {1, 2, 3, 4}
set2 = {1, 2}
print(set1.issuperset(set2))
```

```
'''
```

**Salida:**

```
...
True
...
```

---

## **`union()`**

El método ``union()`` devuelve un nuevo conjunto con todos los elementos de los conjuntos originales.

### **Ejemplo:**

```
```python
set1 = {1, 2, 3}
set2 = {3, 4, 5}
print(set1.union(set2))
...

```

Salida:

```
...
{1, 2, 3, 4, 5}
...

```

`difference()`

El método ``difference()`` devuelve un nuevo conjunto con los elementos que están en el conjunto original pero no en el otro conjunto.

Ejemplo:

```
```python
set1 = {1, 2, 3}
set2 = {2, 3, 4}
print(set1.difference(set2))
...

```

### **Salida:**

```
...
{1}

```

```
'''
```

---

## **`difference\_update()`**

El método ``difference_update()`` elimina del conjunto original todos los elementos que están en otro conjunto.

### **Ejemplo:**

```
'''python
set1 = {1, 2, 3}
set2 = {2, 3, 4}
set1.difference_update(set2)
print(set1)
```

```
'''
```

### **Salida:**

```
'''
```

```
{1}
```

```
'''
```

---

## **`intersection()`**

El método ``intersection()`` devuelve un nuevo conjunto con los elementos comunes a ambos conjuntos.

### **Ejemplo:**

```
'''python
set1 = {1, 2, 3}
set2 = {2, 3, 4}
print(set1.intersection(set2))
```

```
'''
```

### **Salida:**

```
'''
```

```
{2, 3}
```

```
'''
```

---

## **`intersection\_update()`**

El método `intersection_update()` actualiza el conjunto original conservando solo los elementos comunes a ambos conjuntos.

### **Ejemplo:**

```
```python
set1 = {1, 2, 3}
set2 = {2, 3, 4}
set1.intersection_update(set2)
print(set1)

...`
```

Salida:

```
...
{2, 3}
...`
```

Métodos de Diccionarios en Python

Explicación de los métodos de diccionarios

Los diccionarios en Python son colecciones de pares clave-valor que permiten almacenar datos de manera eficiente. A continuación se explican algunos de los métodos más comunes para trabajar con diccionarios junto con ejemplos prácticos de uso en código.

`get()`

El método `get()` devuelve el valor de la clave especificada. Si la clave no se encuentra en el diccionario, devuelve un valor por defecto (opcional).

Ejemplo:

```
```python
colores = {"amarillo": "yellow", "azul": "blue", "verde": "green"}
print(colores.get("amarillo"))
print(colores.get("rojo", "no hay clave rojo"))

...`
```

**Salida:**

```
...
yellow
no hay clave rojo
...
```

---

**`keys()`**

El método ``keys()`` devuelve una vista de todas las claves en el diccionario.

**Ejemplo:**

```
```python
colores = {"amarillo": "yellow", "azul": "blue", "verde": "green"}
print(colores.keys())
...

```

Salida:

```
...
dict_keys(['amarillo', 'azul', 'verde'])
...

```

`values()`

El método ``values()`` devuelve una vista de todos los valores en el diccionario.

Ejemplo:

```
```python
colores = {"amarillo": "yellow", "azul": "blue", "verde": "green"}
print(colores.values())
...

```

**Salida:**

```
...
dict_values(['yellow', 'blue', 'green'])

```



...

---

## items()

El método `items()` devuelve una vista de todos los pares clave-valor en el diccionario como tuplas.

### Ejemplo:

```
colores = {"amarillo": "yellow", "azul": "blue", "verde": "green"}
print(colores.items())
```

### Salida:

```
dict_items([('amarillo', 'yellow'), ('azul', 'blue'), ('verde', 'green')])
```

### Ejemplo con iteración:

```
colores = {"amarillo": "yellow", "azul": "blue", "verde": "green"}
for clave, valor in colores.items():
 print(f"La clave es {clave} y el valor es {valor}")
```

### Salida:

```
La clave es amarillo y el valor es yellow
La clave es azul y el valor es blue
La clave es verde y el valor es green
```

---

# Operadores Básicos en Python

## Introducción a los Operadores Básicos

En Python, los operadores se utilizan para realizar operaciones sobre valores y variables. Los operadores básicos en Python se dividen en tres categorías principales: aritméticos, lógicos y relacionales. A continuación se explican cada uno de estos operadores junto con ejemplos de uso en código.

### Operadores Aritméticos

Los operadores aritméticos se utilizan para realizar operaciones matemáticas comunes.

- **Suma (+):** Suma dos operandos.

```
a = 5
b = 3
print(a + b) # Salida: 8
```

- **Resta (-):** Resta el segundo operando del primero.

```
a = 5
b = 3
print(a - b) # Salida: 2
```

- **Multiplicación (\*):** Multiplica dos operandos.

```
a = 5
b = 3
print(a * b) # Salida: 15
```

- **División (/):** Divide el primer operando por el segundo. El resultado es un número flotante.

```
a = 5
b = 3
print(a / b) # Salida: 1.6666666666666667
```

- **División Entera (/ /):** Divide el primer operando por el segundo y devuelve la parte entera del resultado.

```
a = 5
b = 3
print(a // b) # Salida: 1
```

- **Módulo (%):** Devuelve el resto de la división del primer operando por el segundo.

```
a = 5
b = 3
```

```
print(a % b) # Salida: 2
```

- **Potencia (\*\*):** Eleva el primer operando a la potencia del segundo.

```
a = 5
b = 3
print(a ** b) # Salida: 125
```

---

## Operadores Lógicos

Los operadores lógicos se utilizan para combinar declaraciones condicionales.

- **AND (and):** Devuelve **True** si ambos operandos son **True**.

```
a = True
b = False
print(a and b) # Salida: False
```

- **OR (or):** Devuelve **True** si al menos uno de los operandos es **True**.

```
a = True
b = False
print(a or b) # Salida: True
```

- **NOT (not):** Invierte el valor lógico del operando.

```
a = True
print(not a) # Salida: False
```

---

## Operadores Relacionales

Los operadores relacionales se utilizan para comparar dos valores.

- **Igual a (==):** Devuelve **True** si ambos operandos son iguales.

```
a = 5
b = 3
```

```
print(a == b) # Salida: False
```

- **Distinto de (!=):** Devuelve **True** si ambos operandos no son iguales.

```
a = 5
b = 3
print(a != b) # Salida: True
```

- **Mayor que (>):** Devuelve **True** si el primer operando es mayor que el segundo.

```
a = 5
b = 3
print(a > b) # Salida: True
```

- **Menor que (<):** Devuelve **True** si el primer operando es menor que el segundo.

```
a = 5
b = 3
print(a < b) # Salida: False
```

- **Mayor o igual que (>=):** Devuelve **True** si el primer operando es mayor o igual que el segundo.

```
a = 5
b = 3
print(a >= b) # Salida: True
```

- **Menor o igual que (<=):** Devuelve **True** si el primer operando es menor o igual que el segundo.

```
a = 5
b = 3
print(a <= b) # Salida: False
```

---

## Resumen

Los operadores básicos en Python permiten realizar operaciones matemáticas, lógicas y comparativas de manera eficiente. Los operadores aritméticos incluyen suma, resta, multiplicación, división, división entera, módulo y potencia. Los operadores lógicos permiten combinar condiciones utilizando AND, OR y NOT. Los operadores relacionales permiten comparar valores utilizando igual a, distinto de, mayor que, menor que, mayor o igual que, y menor o igual que. Estos operadores son fundamentales para el desarrollo de cualquier programa en Python.

---

# Expresiones Anidadas y Precedencia de Operadores en Python

En Python, las expresiones anidadas son combinaciones de múltiples operadores y operandos que se evalúan siguiendo ciertas reglas de precedencia. La comprensión de estas reglas es crucial para evitar errores y obtener los resultados esperados.

## Reglas de Precedencia de Operadores

Las reglas de precedencia determinan el orden en el que se evalúan los operadores en una expresión. Los operadores con mayor precedencia se evalúan antes que los de menor precedencia. A continuación se detallan las reglas de precedencia de operadores en Python, de mayor a menor precedencia:

- **Paréntesis ( )**: Las expresiones entre paréntesis se evalúan primero.

```
resultado = (2 + 3) * 4 # Salida: 20
```

- **Exponentes \*\***: Se evalúan a continuación.

```
resultado = 2 * 3 ** 2 # Salida: 18
```

- **Operadores Unarios (+x, -x)**: Se evalúan después.

```
resultado = -3 + 5 # Salida: 2
```

- **Multiplicación, División, División Entera y Módulo (\*, /, //, %)**:

```
resultado = 10 // 3 * 2 # Salida: 6
```

- **Suma y Resta (+, -):**

```
resultado = 3 + 5 - 2 # Salida: 6
```

- **Operadores Relacionales (<, <=, >, >=):**

```
resultado = 3 < 5 # Salida: True
```

- **Igualdad y Desigualdad (==, !=):**

```
resultado = 3 == 3 # Salida: True
```

- **Operadores Lógicos (not, and, or):** Se evalúan en ese orden.

```
resultado = not (3 > 5 and 2 < 4) or 4 == 4 # Salida: True
```

---

## Evaluación de Expresiones Anidadas

Las expresiones anidadas combinan múltiples operadores y operandos. Python evalúa estas expresiones siguiendo las reglas de precedencia y la asociatividad de los operadores (de izquierda a derecha para la mayoría, de derecha a izquierda para exponentes).

### Ejemplo:

```
resultado = 3 + 5 * 2 ** 2 - (4 / 2)
Paso 1: (4 / 2) → 2.0
Paso 2: 2 ** 2 → 4
Paso 3: 5 * 4 → 20
Paso 4: 3 + 20 - 2.0 → 21.0
print(resultado)
```

## Evitar Errores Comunes

1. **Uso de Paréntesis:**

resultado = (3 + 5) \* (2 - 4 / 2) # Salida: -8.0

2. **Comprender la Precedencia y Asociatividad:** Ayuda a predecir el resultado de expresiones complejas.

3. **División Entera vs. Flotante:**

a = 7

b = 3

print(a // b) # Salida: 2

print(a / b) # Salida: 2.3333333333333335

---

## Resumen

Las expresiones anidadas y la precedencia de operadores son conceptos fundamentales en Python que determinan cómo se evalúan las expresiones complejas. Conocer las reglas de precedencia y utilizar paréntesis adecuadamente ayuda a evitar errores y a escribir código más claro y legible.

---

### Material complementario:

- [Operadores relacionales](#)
- [Operadores lógicos](#)
- [Operadores en Python - FreeCodeCamp](#)

### Google Colab:

- [Ejemplo de clase en Colab](#)